

Ciclo II

Desarrollo de Aplicaciones Web

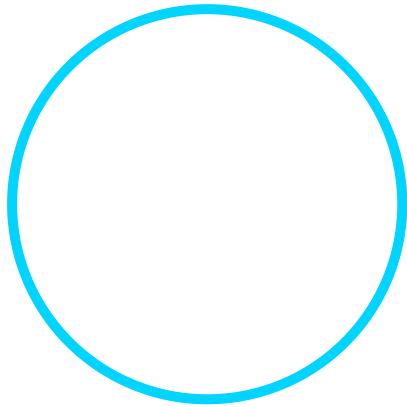
Software Interpretado en el Cliente

Mg. Dennis Antonio Pérez Escobar

Agenda de la Sesión

- | Perfil del Docente y Dinámica Inicial
- | Lineamientos y Fechas Estratégicas
- | Proyectos de Cátedra
- | 1.1. Introducción y Contexto de JavaScript
- | 1.2. Ubicación y Vinculación de Scripts
- | 1.3. Variables, Tipos y Matrices de Decisión
- | 1.4. Estructuras de Control de Flujo

Conociendo al Docente



Mg. Dennis Pérez

- | Máster en Arquitectura de Software e Ing. en Ciencias de la Computación (UDB).
- | CEO y Fundador de Tecnova Sv.
- | Especialista en soluciones Full-Stack (.NET Core, SQL Server, Angular, Node.js).
- | Consultor de automatizaciones empresariales, arquitecturas distribuidas e IA.

Timeline de Evaluaciones (Primer Registro)

30/07/2026

Entrega de Investigación Documental Avanzada (10% de la nota de registro).

15/08/2026

Entrega de la 1era Fase del Proyecto Práctico de Cátedra (15% de la nota de registro).

Proyectos de Cátedra (Vanilla JS Estricto)

Regla base: Cero Frameworks o Librerías. Uso nativo del DOM y XMLHttpRequest.

?? Simulador de Carrito

E-commerce completo a partir de JSON local, cálculos financieros precisos y persistencia en localStorage.

?? Dashboard de Clima

Consumo asíncrono de APIs externas mediante AJAX y manejo estricto de códigos de respuesta HTTP.

? Gestor de Tareas Sync

Flujo CRUD simulado con respuestas Mock API y transformación reactiva del árbol del DOM.

?? Trivia interactiva

Motor de evaluación dinámica de eventos en tiempo real y carga asíncrona de bancos de preguntas.

1.1. Contexto Global de JavaScript

De Scripting Local a Omnipresencia

Originalmente concebido en 1995 para validaciones simples de formularios en el cliente, JavaScript ha mutado hasta convertirse en el estándar de software más distribuido globalmente.

Hoy es un lenguaje de programación formal, multiparadigma, de hilo único e impulsado por eventos, cuya velocidad está determinada por la optimización JIT (Just-In-Time) de motores modernos.

Pilares Fundamentales

- Concurrencia basada en un Event Loop.
- Asincronía no bloqueante nativa.
- Tipado dinámico y débil (coerción automática de tipos).

1.1. Campos de Aplicación Industrial

??? Cliente (Frontend)

Manipulación interactiva del DOM, SPAs, aplicaciones empresariales complejas y gestión de estado local.

?? Servidor (Backend)

Entornos de ejecución como Node.js, Bun y Deno. Construcción de APIs RESTful escalables de alta concurrencia.

?? Aplicaciones Móviles

Frameworks híbridos y nativos (React Native, Ionic) para distribución multiplataforma con un solo codebase.

?? Automatizaciones e IA

Orquestación de flujos de automatización (n8n), scripts de integración continua y consumo de agentes de IA.

1.2. Métodos de Inclusión de Código

Existen tres mecánicas para inyectar lógica de JavaScript en un documento de hipertexto. Comprender su impacto arquitectónico define el rendimiento de la carga inicial.

1. Inline (Línea)

Código incrustado directamente dentro de los atributos HTML.

```
<button onclick="alert()">
```

2. Internal (Interno)

Lógica estructurada dentro de etiquetas delimitadoras específicas.

```
<script>
  let x = 10;
</script>
```

3. External (Externo)

Ficheros independientes con extensión `.js` vinculados formalmente.

```
<script src="main.js">
</script>
```


1.2. Benchmark Arquitectónico

Característica	Inline (En Línea)	Internal (Interno)	External (Externo)
Separación de Conceptos (SoC)	Nula	Parcial	Total (Estándar)
Reutilización de Código	Imposible	Solo en el archivo	Global e ilimitada
Aprovechamiento de Caché	No aplica	Inexistente	Excelente (Rendimiento)
Mantenibilidad Enterprise	Crítica	Compleja	Alta eficiencia

1.3. Mutabilidad y Ámbitos de Variables

El estándar ECMA Moderno eliminó el uso de `var` debido a efectos colaterales como el *Hoisting* y la falta de aislamiento de memoria en bloques lógicos.

- | **const:** Define referencias constantes e inmutables. Obligatorio por defecto en arquitecturas modernas.
- | **let:** Define variables de bloque. Solo se usa cuando el algoritmo exige reasignación.

```
// Buenas prácticas corporativas
const IMPUESTO = 0.13;
const apiURL = "https://api.udb.edu";

let totalFactura = 0.0;
let contadorItems = 2;

// var queda estrictamente prohibido
var banderaGlobal = true;
```

1.3. Matriz de Decisión: ¿let o const?

Regla de oro moderna: Usa `const` siempre por defecto. Cambia a `let` únicamente si la lógica del algoritmo exige reasignación explícita.

Criterio Técnico / Impacto	<code>const</code> (Asignación Inmutable)	<code>let</code> (Asignación Mutable)
Reasignación del puntero (=)	Prohibida (Dispara TypeError)	Permitida
Mutabilidad Interna (Objetos/Arrays)	Permitida (Propiedades pueden mutar)	Permitida
Impacto en Memoria (Motor V8)	Máxima optimización (Puntero fijo)	Mayor carga al recolector de basura
Legibilidad para el Equipo	Garantiza que el valor origen no será alterado.	Advierte al equipo que el valor cambiará.
Casos de Uso Industriales	Llamadas a APIs, nodos del DOM, configuraciones.	Contadores (for), acumuladores matemáticos, flags.

1.3. Contexto Real de Tipos de Datos

En entornos corporativos, la correcta elección del tipo primitivo previene errores lógicos graves durante el procesamiento transaccional.

?? String (Cadenas)

Identificadores únicos, nombres de clientes o códigos UUID.

```
const codigoProducto = "PRD-7161";
```

?? Number (Numéricos)

Cálculos de montos, tasas de descuento o contadores transaccionales.

```
let subtotalFactura = 249.50;
```

?? Boolean (Lógicos)

Banderas de control de estado, validación de accesos o sesiones.

```
let esContribuyente = true;
```

??? Undefined & Null

Manejo explícito de respuestas vacías o datos aún no inicializados.

```
let datosBancarios = null; // Vacío intencional
```

1.4. Estructuras de Control: Condicionales

Las sentencias condicionales permiten evaluar bifurcaciones de lógica basadas en expresiones Booleanas estrictas.

- | **if:** Evaluación única.
- | **if-else:** Flujo binario excluyente.
- | **if-else if-else:** Múltiples condiciones secuenciales jerárquicas.

```
if (montoCompra >= 500) {
    aplicarDescuentoMayoreo();
} else if (montoCompra >= 200) {
    aplicarDescuentoMinorista();
} else {
    cobrarTarifaEstandar();
}
```

1.4. Evaluación en Cortocircuito

Mecánica optimizada donde el motor detiene la evaluación lógica en el momento en que el resultado final está garantizado.

Operador OR (||)

Retorna el primer valor evaluado como "Verdadero" (Truthy). Ideal para contingencia de datos.

```
let nombreUser = input || "Anónimo";
```

Operador AND (&&)

Evalúa el segundo operando solo si el primero es verdadero. Excelente como guarda de seguridad.

```
tokenActivo && renderizarModulo();
```

1.4. Sentencia Switch y Anidaciones

La estructura `switch` evalúa de forma estricta una sola variable contra múltiples escenarios numéricos o de texto.

Peligro de Cascada

Omitir el comando `break;` provocará la ejecución continua de los bloques inferiores.

```
switch(estadoFactura) {  
    case 'PAGADA':  
        liberarServicio();  
        break;  
    case 'PROCESANDO':  
        mostrarLoader();  
        break;  
    default:  
        bloquearAcceso();  
}
```

¿Preguntas?

Próxima Sesión: Interacción con el DOM y Manejo de Eventos

Contacto: **dennis.perez@udb.edu.sv**